2

Nomes, vinculações e escopos

Universidade Estadual da Paraíba (UEPB)
Paradigmas de Programação
Kláudio Henrique Mascarenhas Medeiros

Nome (identificador)

- O nome (comumente chamado de identificador) é uma cadeia de caracteres utilizado para a identificação de uma entidade em um programa.

Nome (identificador)

- O **nome** (comumente chamado de **identificador**) é uma cadeia de caracteres utilizado para a identificação de uma entidade em um programa.

- Algumas questões envolvendo nome são:
  - os nomes podem ter caracteres especiais?

Nome (identificador)

- O **nome** (comumente chamado de **identificador**) é uma cadeia de caracteres utilizado para a identificação de uma entidade em um programa.

- Algumas questões envolvendo nome são:
  - os nomes podem ter caracteres especiais?
  - os nomes distinguem maiúsculo de minúsculo?

Nome (identificador)

A maioria das linguagens que vimos até agora provavelmente é *case-sensitive*. Pascal é uma linguagem *case-insensitive*.

Nome (identificador)

```
program Hello;
var numero1: Integer;
begin
    numero1 := 10;
    writeln('This is an output');
    Writeln(Numero1);
end.
```

Código Pascal para definir e imprimir alguns valores. Perceba que numero1 e Numero1 podem ser utilizados dizendo respeito à mesma coisa.

Nome (identificador)

- O **nome** (comumente chamado de **identificador**) é uma cadeia de caracteres utilizado para a identificação de uma entidade em um programa.

- Algumas questões envolvendo nome são:
  - os nomes podem ter caracteres especiais?
  - os nomes distinguem maiúsculo de minúsculo?
  - os nomes possuem um tamanho limite?

Nome (identificador)

- Comumente, identificadores começam com uma letra (a-z), um *underline* (_) seguido de uma letra ou algum dígito.

Nome (identificador)

- Comumente, identificadores começam com uma letra (a-z), um *underline* (_) seguido de uma letra ou algum dígito.

- Exemplo: em algumas linguagens, o "_" é utilizado antes para indicar um campo ou método privado.

Link: https://www.geeksforgeeks.org/underscore-_-python/

Nome (identificador)

Em Python, o *underline* antes do nome de uma variável faz com que ela seja acessível apenas no módulo em que foi definida (“privada”)

Palavra reservada e palavra-chave

- Estes são termos levemente distintos:
    - As **palavras-chave** (*keywords*) são palavras da linguagem que são especiais em certos contextos.

Palavra reservada e palavra-chave

- Estes são termos levemente distintos:
  - As **palavras-chave** (*keywords*) são palavras da linguagem que são especiais em certos contextos.
  - As **palavras reservadas** são palavras que são proibidas de serem utilizadas como identificadores.

Palavra reservada e palavra-chave

- Exemplo: a palavra **value** em C# não é reservada em certos contextos (são em propriedades).

Palavra reservada e palavra-chave

- Exemplo: a palavra **value** em C# não é reservada em certos contextos (são em propriedades).

```
public class WordingTest
{
    private int number;

    public int prop
    {
        get {return this.number;}
        set {this.number = value;}
    }
}
```

```
WordingTest w = new WordingTest();
w.prop = 10;
Debug.Log(w.prop);

int value = 20;
Debug.Log(value);
```

Palavra reservada e palavra-chave

- As palavras reservadas tendem a ser mais interessantes que as palavras-chave no quesito projeto de linguagem. Qual seria uma justificativa favorável para essa afirmação?

Palavra reservada e palavra-chave

- As palavras reservadas tendem a ser mais interessantes que as palavras-chave no quesito projeto de linguagem. Qual seria uma justificativa favorável para essa afirmação?

- Saber quando uma palavra tem ou não um significado especial **pode ser bastante confuso**.

Palavra reservada e palavra-chave

- As palavras reservadas tendem a ser mais interessantes que as palavras-chave no quesito projeto de linguagem. Qual seria uma justificativa favorável para essa afirmação?

- Saber quando uma palavra tem ou não um significado especial **pode ser bastante confuso**.

- A definição de um Real Integer* ou Integer Real em Fortran (*variável Integer do tipo Real).

Variável

- Uma **variável** é uma abstração de uma célula de memória do computador.

Variável

- Uma **variável** é uma abstração de uma célula de memória do computador.

- A variável possui um **nome**, um **endereço de memória**, um **valor** (armazenado em um dado momento), **tempo de vida** (tempo no qual a memória está alocada para a variável) e **escopo** (locais do programa onde a variável é conhecida).

Variável

- Em C...
  - podemos acessar o **endereço de memória** de uma variável utilizando &.

Variável

- Em C...
  - podemos acessar o **endereço de memória** de uma variável utilizando &.
  - o **escopo** de uma variável definida dentro do script e em uma função dentro do script é diferente.

Variável

- Em C...
  - podemos acessar o **endereço de memória** de uma variável utilizando &.
  - o **escopo** de uma variável definida dentro do script e em uma função dentro do script é diferente.
  - existem diferentes **tempos de vida** para chamadas recursivas de uma função (e.g. fatorial).

Variável

```c
#include <stdio.h>
int var = 10;

float mult(float op1, float op2)
{
    float result = op1 * op2;
    return result;
}
int main()
{
    printf("%d\n", var);
    printf("%d\n", result); // não compilará
}
```

Variável

```
#include <stdio.h>

int main()
{
    int num = 10;
    printf("%d %d\n", num, &num);

}
```

Variável

- Pode acontecer de várias variáveis possuírem o mesmo endereço. Elas são **apelidos** (*aliases*).

Variável

- Pode acontecer de várias variáveis possuírem o mesmo endereço. Elas são **apelidos** (*aliases*).

- Podemos, através de uma, “mudar” a outra. É necessário cuidado com essas situações.

Variável

```
lista1 = [1, 2, 3]
lista2 = lista1
lista1.append(4)
print(lista2)# imprimirá [1, 2, 3, 4]
```

Variável

```c
#include <stdio.h>

int main()
{
    int var;
    int *varPtr = &var;
    int *varPtr2 = &var;
    var = 5;
    printf("%d %d %d\n", var, *varPtr, *varPtr2);
    *varPtr = 8;
    printf("%d %d %d\n", var, *varPtr, *varPtr2);
    // var, *varPtr e *varPtr2 são apelidos (aliases)
}
```

Vinculação (*binding*)

- A **vinculação** (*binding*) envolve a associação entre um identificador e sua declaração em um programa.

Vinculação (*binding*)

- A **vinculação** (*binding*) envolve a associação entre um identificador e sua declaração em um programa.

- A vinculação pode significar a atribuição de um valor para a locação, nome, valor ou tipo de uma variável.

Vinculação (*binding*)

- A **vinculação** (*binding*) envolve a associação entre um identificador e sua declaração em um programa.

- A vinculação pode significar a atribuição de um valor para a locação, nome, valor ou tipo de uma variável.

- As vinculações ocorrem em tempos distintos, podendo ser, por exemplo, em compilação, carga ou execução.

Vinculação (*binding*)

- Exemplo: em uma instrução “A = B + C”, comumente:
    - se obtém os endereços de A, B e de C
    - são obtidos os dados dos endereços de B e de C
    - é computado o resultado de B + C
    - o resultado computado é atribuído à locação de A

Vinculação (*binding*)

- Exemplo: em uma instrução “n = n + 10;” em Java:
    - tipo de n é vinculado em tempo de compilação;

Vinculação (*binding*)

- Exemplo: em uma instrução “n = n + 10;” em Java:
  - **tipo** de n é vinculado em tempo de compilação;
  - o **conjunto de valores possíveis** de n é vinculado em tempo de carga;

Vinculação (*binding*)

- Exemplo: em uma instrução “n = n + 10;” em Java:
  - **tipo** de n é vinculado em tempo de compilação;
  - o **conjunto de valores possíveis** de n é vinculado em tempo de carga;
  - o **significado de ‘+’** é vinculado em tempo de compilação, após os tipos dos operandos determinados

Vinculação (*binding*)

- Exemplo: em uma instrução “n = n + 10;” em Java:
  - **tipo** de n é vinculado em tempo de compilação;
  - o **conjunto de valores possíveis** de n é vinculado em tempo de carga;
  - o **significado de ‘+’** é vinculado em tempo de compilação, após os tipos dos operandos determinados
  - a **representação do literal 5** é vinculada em tempo de carga

Vinculação (*binding*)

- Exemplo: em uma instrução “n = n + 10;” em Java:
  - **tipo** de n é vinculado em tempo de compilação;
  - o **conjunto de valores possíveis** de n é vinculado em tempo de carga;
  - o **significado de ‘+’** é vinculado em tempo de compilação, após os tipos dos operandos determinados
  - a **representação do literal 5** é vinculada em tempo de carga
  - o **valor de n** é vinculado em tempo de execução

Vinculação (*binding*)

- As **vinculações de locação** costumam ser em tempo de carga, podendo acontecer depois (e.g. variáveis em procedimentos ou alocação dinâmica).

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

struct node {
    float num;
    struct node *nextPtr;
};

int main()
{
    struct node *firstNode = NULL;
    srand(time(NULL));
    int value = rand() % 2;
    if(value)
    {
        firstNode = (struct node*) malloc(sizeof(struct node));
        firstNode -> num = 4;
    }
    if(firstNode != NULL)
        printf("%.2f\n", firstNode -> num);
    else
        printf("The node is null");
}
```

Vinculação (*binding*)

- As **vinculações de locação** costumam ser em tempo de carga, podendo acontecer depois (e.g. variáveis em procedimentos ou alocação dinâmica).

- As **vinculações de nome** costumam ser em tempo de compilação (quando a declaração é encontrada).

Vinculação (*binding*)

- As **vinculações de locação** costumam ser em tempo de carga, podendo acontecer depois (e.g. variáveis em procedimentos ou alocação dinâmica).

- As **vinculações de nome** costumam ser em tempo de compilação (quando a declaração é encontrada).

- As **vinculações de valor** costumam ser em tempo de execução.

Vinculação (*binding*)

- As **vinculações de tipo** costumam ser em tempo de compilação (pelas declarações de tipo), mas podem também acontecer em tempo de execução.
    - **estáticas**: antes do tempo de execução e inalteradas durante a execução.

Vinculação (*binding*)

- As **vinculações de tipo** costumam ser em tempo de compilação (pelas declarações de tipo), mas podem também acontecer em tempo de execução.
  - **estáticas**: antes do tempo de execução e inalteradas durante a execução.
    - explícita: e.g. int x;
    - implícita: alguma convenção p/ determinação (e.g. Perl usa $ p/ escalar, @ p/ vetor, etc)

Vinculação (*binding*)

- As **vinculações de tipo** costumam ser em tempo de compilação (pelas declarações de tipo), mas podem também acontecer em tempo de execução.
    - **dinâmicas**: durante a execução do programa, podendo ser durante ela alterada.

Vinculação (*binding*)

- As **vinculações de tipo** costumam ser em tempo de compilação (pelas declarações de tipo), mas podem também acontecer em tempo de execução.
    - **dinâmicas**: durante a execução do programa, podendo ser durante ela alterada.
    - no caso, a variável é vinculada a um tipo quando recebe um valor (e.g. “x = 10” em Python)

Vinculação e tempo de vida

- Em linguagens imperativas, existe a noção de alocação e de liberação.

Vinculação e tempo de vida

- Em linguagens imperativas, existe a noção de alocação e de liberação.

- A **alocação** está relacionada com uma célula de memória dentre células disponíveis a qual uma variável é vinculada.

Vinculação e tempo de vida

- Em linguagens imperativas, existe a noção de alocação e de liberação.

- A **alocação** está relacionada com uma célula de memória dentre células disponíveis a qual uma variável é vinculada.

- A **liberação** está relacionada com a devolução da célula de memória não mais necessária pela variável.

Vinculação e tempo de vida

Em C, podemos alocar memória dinamicamente com malloc() e liberar memória alocada com free()

Vinculação e tempo de vida

- O **tempo de vida** de uma variável diz respeito ao tempo em que ela fica vinculada a uma posição específica da memória do computador.

Vinculação e tempo de vida

- O **tempo de vida** de uma variável diz respeito ao tempo em que ela fica vinculada a uma posição específica da memória do computador.

- Podem ser estáticas, dinâmicas da pilha, dinâmicas do monte explícitas e dinâmicas do monte implícitas.

Tempos de vida

- As **variáveis estáticas** são vinculadas a células de memória antes da execução. Essa vinculação persiste durante toda a execução do programa.

Tempos de vida

- As **variáveis estáticas** são vinculadas a células de memória antes da execução. Essa vinculação persiste durante toda a execução do programa.

- Como vantagens, eficiência (pelo endereçamento direto) e não há sobrecarga em tempo de execução para liberação e alocação.

Tempos de vida

- As **variáveis estáticas** são vinculadas a células de memória antes da execução. Essa vinculação persiste durante toda a execução do programa.

- Como vantagens, eficiência (pelo endereçamento direto) e não há sobrecarga em tempo de execução para liberação e alocação.

- Como desvantagens, falta de flexibilidade e armazenamento não compartilhado entre variáveis.

Link:https://www.geeksforgeeks.org/static-variables-in-c/

Tempos de vida

Exemplo: variáveis estáticas em C

```
#include <stdio.h>
void f()
{
     static int x;
     x++;
     printf("%d\n", x);
}
int main()
{
     for(int i = 0; i < 10; i++){
          f();
     }

}
```

x preserva seu valor mesmo após a função acabar. As variáveis C podem estar em diferentes locais, como na heap (para memória alocada dinamicamente), a pilha (para chamadas recursivas) ou o segmento de dados (data segment, para variáveis static)

Link:https://www.geeksforgeeks.org/static-variables-in-c/

Tempos de vida

Exemplo: variáveis estáticas em C

```
#include <stdio.h>
void f()
{
    static int x;
    x++;
    printf("%d\n", x);
}
int main()
{
    for(int i = 0; i < 10; i++){
        f();
    }

}
```

x preserva seu valor mesmo após a função acabar. As variáveis C podem estar em diferentes locais, como na heap (para memória alocada dinamicamente), a pilha (para chamadas recursivas) ou o segmento de dados (data segment, para variáveis static)

É importante constar que x NÃO é uma variável global. Ela continua existindo após o fim da função, mas o escopo dela é local! Com ela, os conceitos de escopo e tempo de vida distinguem de forma mais evidente.

Tempos de vida

- As **variáveis dinâmicas de pilha** são vinculadas quando as sentenças de declaração são elaboradas, mas com tipos estaticamente vinculados.

Tempos de vida

- As **variáveis dinâmicas de pilha** são vinculadas quando as sentenças de declaração são elaboradas, mas com tipos estaticamente vinculados.

- Como vantagens, permite a recursão e otimização do uso do espaço em memória.

Tempos de vida

- As **variáveis dinâmicas de pilha** são vinculadas quando as sentenças de declaração são elaboradas, mas com tipos estaticamente vinculados.

- Como vantagens, permite a recursão e otimização do uso do espaço em memória.

- Como desvantagens, existe uma maior sobrecarga de procedimentos de alocação e liberação.

Tempos de vida

- Um exemplo: imagine uma função f() que contém uma variável inteira num. Além disso, existe um main(), que chama f().

Tempos de vida

- Um exemplo: imagine uma função f() que contém uma variável inteira num. Além disso, existe um main(), que chama f().

- Quando f() começa e a declaração de num é executada, num é vinculada na pilha.

Tempos de vida

- Um exemplo: imagine uma função f() que contém uma variável inteira num. Além disso, existe um main(), que chama f().

- Quando f() começa e a declaração de num é executada, num é vinculada na pilha.

- Quando f() é encerrada, num é liberada da pilha.

Tempos de vida

Exemplo: variável f dentro de fatorial em C

```
#include <stdio.h>
int fatorial(int n)
{
     int f;
     if(n == 1)
          f = 1;
     else
          f = n * fatorial(n - 1);
     return f;
}
int main()
{
     printf("%d\n", fatorial(5));

}
```

Cada chamada de fatorial implicará na criação de uma variável f. Os valores de f serão diferentes em cada chamada.

Tempos de vida

- As **variáveis dinâmicas do monte explícitas** são células de memória não nomeadas alocadas e liberadas por instruções explícitas, em tempo de execução.

Tempos de vida

- As **variáveis dinâmicas do monte explícitas** são células de memória não nomeadas alocadas e liberadas por instruções explícitas, em tempo de execução.

- Essas variáveis são acessíveis apenas através de ponteiros ou outras variáveis de referência.

Tempos de vida

- As **variáveis dinâmicas do monte explícitas** são células de memória não nomeadas alocadas e liberadas por instruções explícitas, em tempo de execução.

- Essas variáveis são acessíveis apenas através de ponteiros ou outras variáveis de referência.

- O monte em questão é o *heap*, coleção de células desorganizadas (imprevisibilidade de uso).

Tempos de vida

- As **variáveis dinâmicas do monte explícitas** são células de memória não nomeadas alocadas e liberadas por instruções explícitas, em tempo de execução.

- Como vantagens, existe armazenamento dinâmico (estruturas que crescem ou diminuem em execução).

Tempos de vida

- As **variáveis dinâmicas do monte explícitas** são células de memória não nomeadas alocadas e liberadas por instruções explícitas, em tempo de execução.

- Como vantagens, existe armazenamento dinâmico (estruturas que crescem ou diminuem em execução).

- Como desvantagens, baixa confiabilidade (o programador é o responsável por gerenciar memória).

Tempos de vida

Exemplo: ponteiros, alocação e liberação em C++

```
...
int *intptr;
intptr = new int;
...
delete intptr;
```

- intptr cria o ponteiro para int
- new int cria uma variável (dinâmica do monte)
- delete intptr libera a variável apontada por intptr

Tempos de vida

Exemplo: objetos em Java

```
...
Classe c = new Classe();
...
```

- Os objetos são dinâmicos do monte explícitos, acessados por variáveis de referência.
- Em Java, não há liberação explícita, ficando essa a cargo do coletor de lixo (*garbage collector*), implícito.

Tempos de vida

- As **variáveis dinâmicas do monte implícitas** são vinculadas ao armazenamento no monte apenas quando são atribuídos valores para elas.

Tempos de vida

- As **variáveis dinâmicas do monte implícitas** são vinculadas ao armazenamento no monte apenas quando são atribuídos valores para elas.

- Como vantagens, provêm uma considerável flexibilidade (como veremos no exemplo).

Tempos de vida

- As **variáveis dinâmicas do monte implícitas** são vinculadas ao armazenamento no monte apenas quando são atribuídos valores para elas.

- Como vantagens, provêm uma considerável flexibilidade (como veremos no exemplo).

- Como desvantagens, gera uma sobrecarga em tempo de execução.

Tempos de vida

Exemplo: variáveis em JavaScript

```
...
valor = [1, 10, 5, 4]
...
```

- A variável valor é um vetor de 4 elementos.
- Não importa se, previamente, a variável “valor” foi usada ou qual era o seu valor anterior.

Link: https://www.baeldung.com/java-stack-heap

Vinculação em Java

- As variáveis de tipos primitivos Java ficam na pilha (*stack*).

Link: https://www.baeldung.com/java-stack-heap

Vinculação em Java

- As **variáveis de tipos primitivos** Java ficam na pilha (*stack*).

- As **referências para os objetos** também ficam na pilha (*stack*). Os **objetos** em si ficam na seção de variáveis dinâmicas do monte (*heap*).

Link: https://www.baeldung.com/java-stack-heap

Vinculação em Java

- As **variáveis de tipos primitivos** Java ficam na pilha (*stack*).

- As **referências para os objetos** também ficam na pilha (*stack*). Os **objetos** em si ficam na seção de variáveis dinâmicas do monte (*heap*).

- O acesso à stack é comumente mais rápido que o à heap. Cada thread tem sua stack. As variáveis criadas em métodos existem enquanto ele estiver rodando.

Link: https://www.baeldung.com/java-stack-heap

Vinculação em Java

- A memória heap de Java não é desalocada de forma automática, havendo um GC (*garbage collector*) que faz isso.

Link: https://www.baeldung.com/java-stack-heap

Vinculação em Java

- A memória heap de Java não é desalocada de forma automática, havendo um GC (*garbage collector*) que faz isso.

- O GC limpará da memória, por exemplo, objetos que não são mais utilizados.

Link: https://www.baeldung.com/java-stack-heap

Vinculação em Java

- A memória heap de Java não é desalocada de forma automática, havendo um GC (*garbage collector*) que faz isso.

- O GC limpará da memória, por exemplo, objetos que não são mais utilizados.

- Enquanto a aplicação estiver rodando, a *heap* existe (diferente da *stack*).

Escopo de variável

- O **escopo** de uma variável corresponde aos trechos de código em que a variável é visível.

Escopo de variável

- O **escopo** de uma variável corresponde aos trechos de código em que a variável é visível.

- A variável é considerada visível na instrução se puder ser acessada pela instrução.

Escopo de variável

- O **escopo** de uma variável corresponde aos trechos de código em que a variável é visível.

- A variável é considerada visível na instrução se puder ser acessada pela instrução.

- Os escopos são comumente determinados por blocos (indentação em Python, chaves em C/Java).

Tempos de vida

Exemplo: trecho de código em C

```
if(cond)
{
    int temp;
    ...
}
```

- A variável temp tem escopo local, sendo definida e visível apenas dentro do bloco delimitado por { e }.

Tempos de vida

Exemplo: trecho de código em C

```
int f(){
    int cont;
    for(...){
        int cont;
    }

}
```

- Esse tipo de trecho é válido em algumas linguagens (e.g. C) e inválido em outras (e.g. Java).

Referências

SEBESTA, Robert M. **Conceitos de Linguagens de Programação**. Bookman, 2018.

Post no StackOverflow sobre diferença entre escopo e tempo de vida: https://pt.stackoverflow.com/questions/135572/qual-a-diferen%C3%A7a-entre-escopo-e-tempo-de-vida